

ID I063 – INTRODUCTION TO PROGRAMMING



Srijith P.K

Position: Associate Professor

Department(s):

- Computer Science & Engineering
- Artificial Intelligence

✉ srijith@cse.iith.ac.in



Saurabh kumar

A-718/A

✉ saurabhkr@cse.iith.ac.in

Cyber Security, Mobile
Security, Cyber forensics and
Malware Analysis.

ADMINs AND TAs

TBD

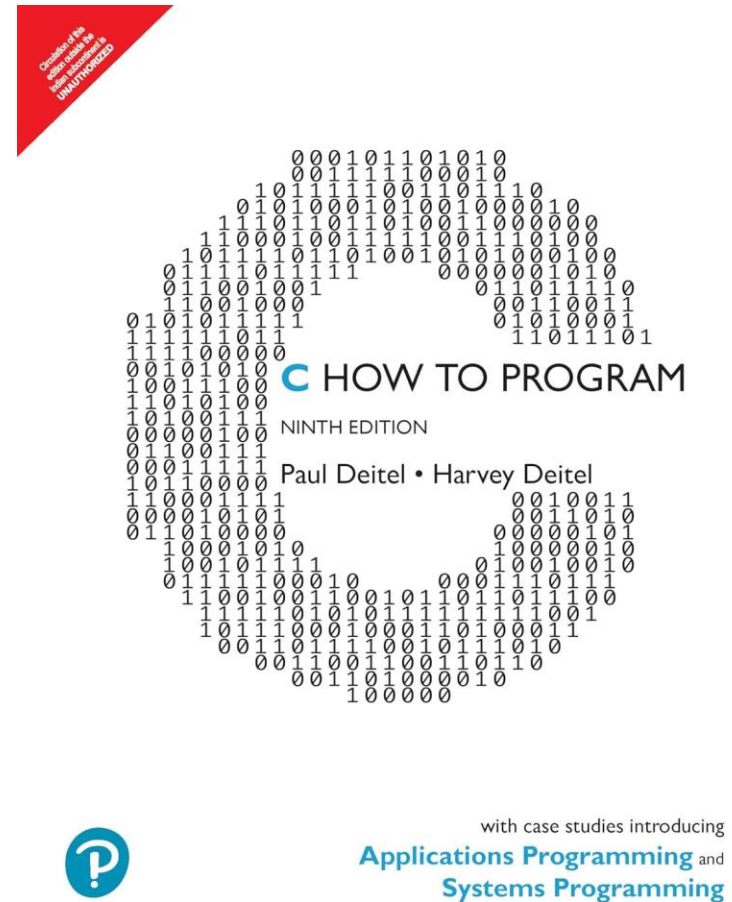
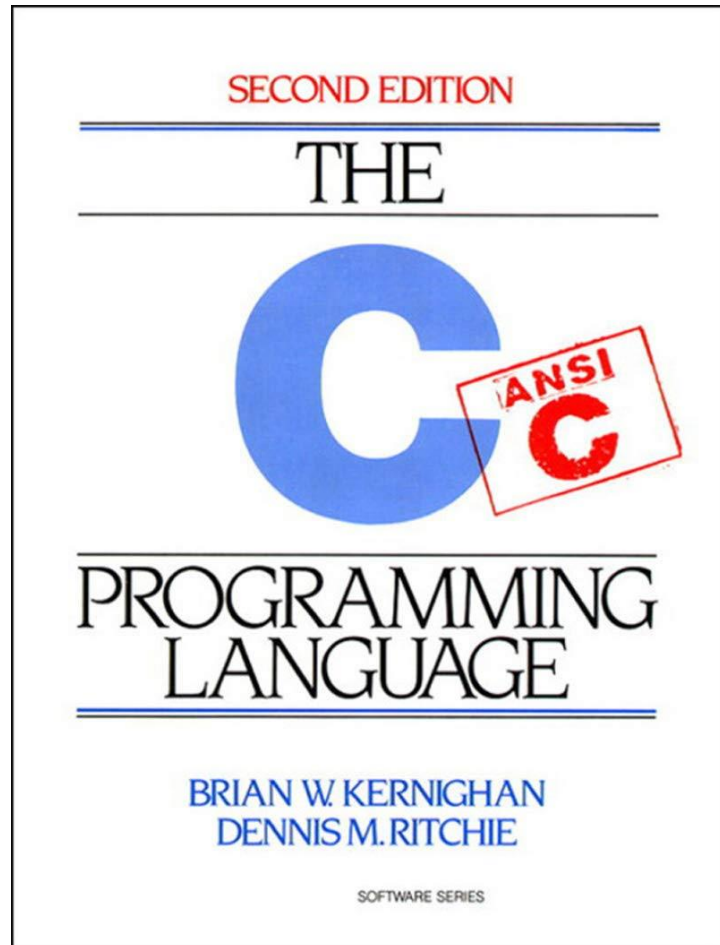
A red forklift is shown in profile, lifting a blue shipping container from a stack. The container is suspended in the air. The background consists of several tall stacks of shipping containers in various colors including blue, orange, red, and green. The sky is blue with some light clouds. The ground is a flat, paved surface.

COURSE LOGISTICS

CONTACT HOURS

- $\approx$ 2 hours of lecture per week
 - Tuesdays 3:00pm-5:00pm @ LHC-5
- $\approx$ 2 hours of hands-on practice per week
 - Thursdays 6pm-8pm @ LHC-5
 - Attendance mandatory

TEXTBOOKS



COURSE GOALS

- Foundations of C language
 - Thinking like a computer programmer
 - You should be able to learn other programming languages on your own
- Write, debug, read C programs
- Good programming practices

COURSE OUTLINE

- Introduction to Computers and C
- Intro to C Programming: Input, Output, Types, Arithmetic
- Structured Program Development and Program control : , if, if/else, while, for, do/while, switch, break, continue, Logical Operators
- Functions
- Arrays
- Pointers
- Characters and Strings
- Formatted Input/Output, Structures, and File Processing

CLASSROOM

- Google Classroom code “wc4u7zcv”





COURSE EVALUATION



EVALUATION SCHEME

- Your programs in weekly practice sessions will be evaluated
 - 15 sessions, each 1 mark – **15 marks**
-
-
-
-
-

EVALUATION SCHEME

- Your programs in weekly practice sessions will be evaluated
 - 15 sessions, each 1 mark – 15 marks
- Alternate week “lab exam”
 - Best 5 of 7 exams, each 5 marks – **25 marks**
-
-
-

EVALUATION SCHEME

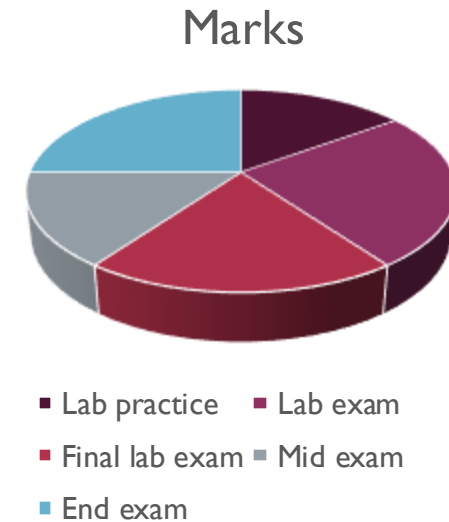
- Your programs in weekly practice sessions will be evaluated
 - 15 sessions, each 1 mark – 15 marks
- Alternate week “lab exam”
 - Best 5 of 7 exams, each 5 marks – 25 marks
- Final lab exam – **20 marks**

■

■

EVALUATION SCHEME

- Your programs in weekly practice sessions will be evaluated
 - 15 sessions, each 1 mark – 15 marks
- Alternate week “lab exam”
 - Best 5 of 7 exams, each 5 marks – 25 marks
- Final lab exam – 20 marks
- Mid-semester written exam – 15 marks
- End-semester written exam – 25 marks



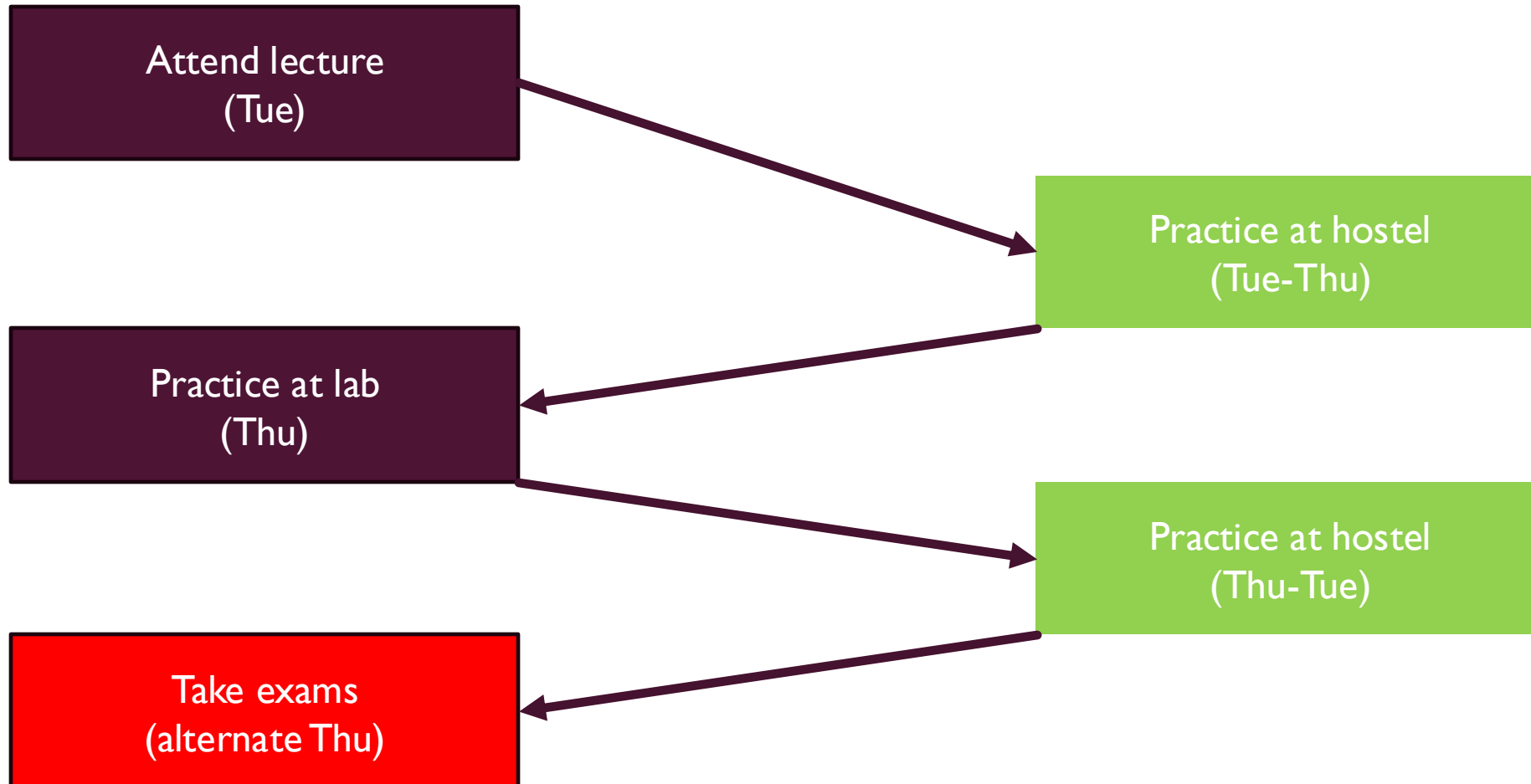
LAB DETAILS

- Every student must bring a **laptop (fully charged)**, **ID card** and a **pen/pencil**
 - No other devices allowed including watches, phones etc. No books, paper etc.
- -
 -
- -
 -
-

LAB DETAILS

- Every student must bring a **laptop (fully charged)**, **ID card** and a **pen/pencil**
 - No other devices allowed including watches, phones etc. No books, paper etc.
- Attempt questions on “**prutor**”
 - Can only be accessed inside the venue, using your login.
 - run our **activity monitor** program
- Practice sessions are collaborative
 - Question briefing will be done
 - TAs may orally quiz randomly
- No collaboration/help in exams

EXPECTED ID I063 STUDENT SCHEDULE 😊





ACADEMIC INTEGRITY AND HONESTY

- IIT expects top standards of integrity
- We have a **plagiarism** policy in place
- Any deviation will lead to “Fail (FR)” grade
 - Case reported to disciplinary action committee



COMPUTER PROGRAMMING

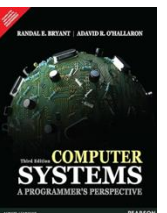
SIMPLIFIED ARCHITECTURE; MACHINE, ASSEMBLY, HIGHER-LEVEL CODING LANGUAGES



DIGITAL COMPUTERS

Computers can perform calculations and make logical decisions phenomenally faster than human beings can. Today's personal computers and smartphones can perform billions of calculations in one second

Computers process data under the control of sequences of instructions called **computer programs** (or simply **programs**).



DIGITAL COMPUTERS

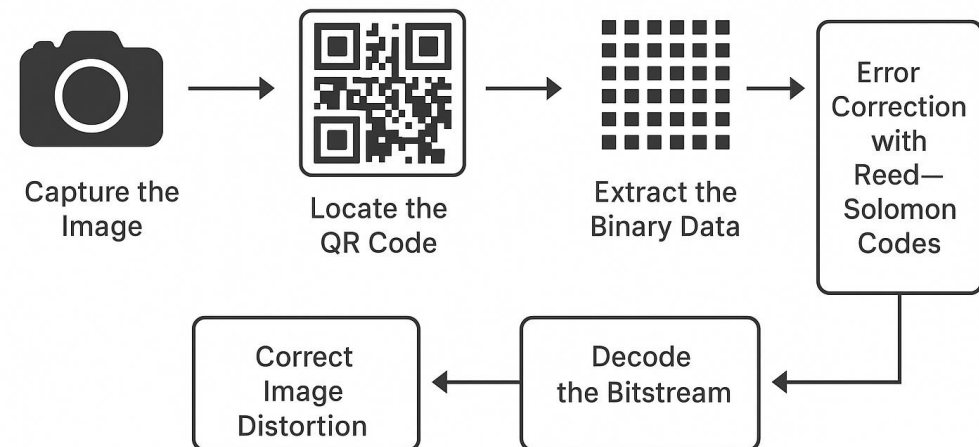
Computers can perform calculations and make logical decisions phenomenally faster than human beings can. Today's personal computers and smartphones can perform billions of calculations in one second

Computers process data under the control of sequences of instructions called **computer programs** (or simply **programs**).

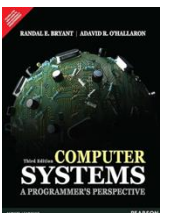
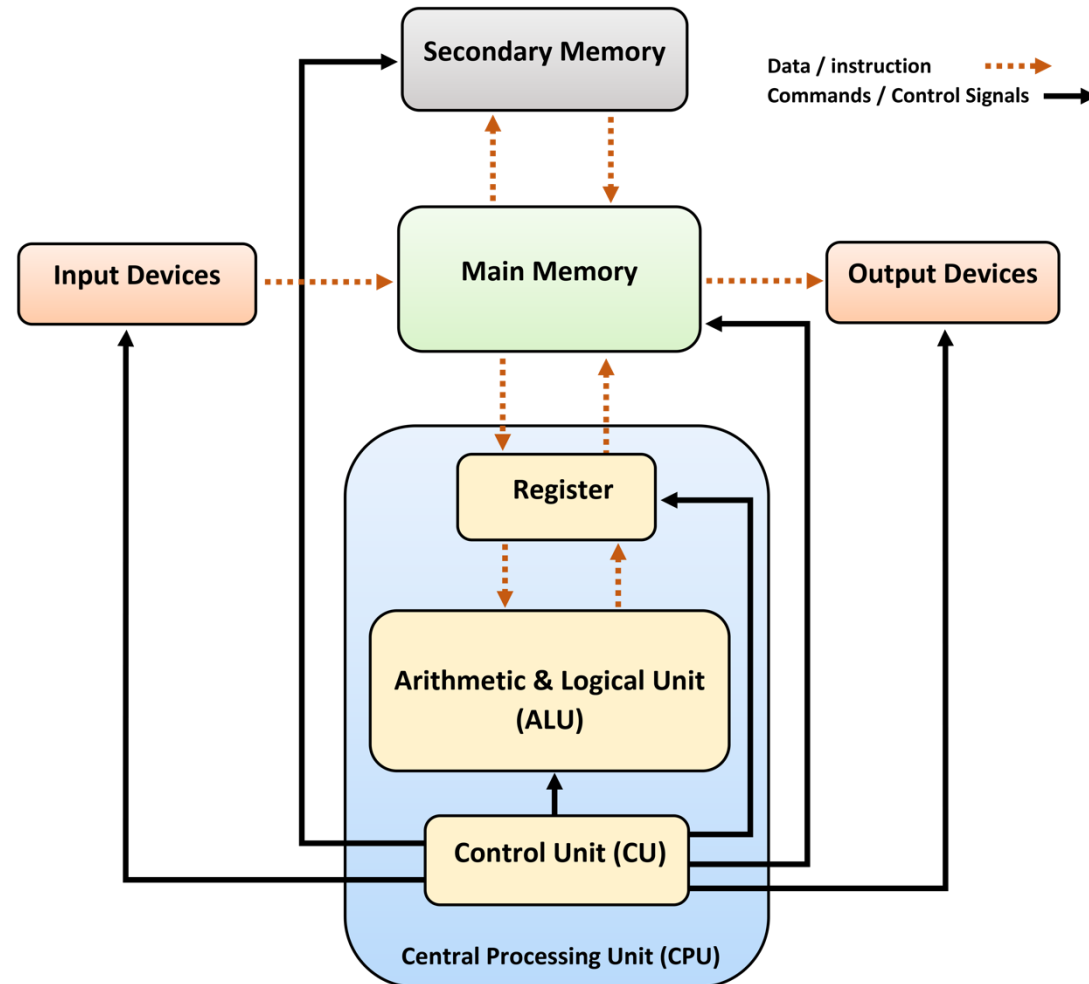
QR code reading involves:

1. Capturing the image
2. Locating and aligning the QR code
3. Extracting binary data from the grid
4. Correcting errors
5. Decoding the bitstream into readable content

How QR Code Reading Works

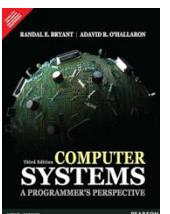


DIGITAL COMPUTERS



DIGITAL COMPUTERS : INPUT

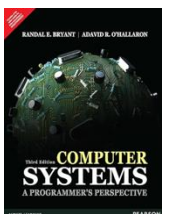
- receiving voice commands,
- scanning images and barcodes,
- reading data from secondary storage devices (such as solid-state drives, hard drives, Blu-ray Disc™ drives and USB flash drives—also called “thumb drives” or “memory sticks”),
- receiving video from a webcam,
- receiving information from the Internet (such as when you stream videos from YouTube® or download e-books from Amazon),
- receiving position data from a GPS device,



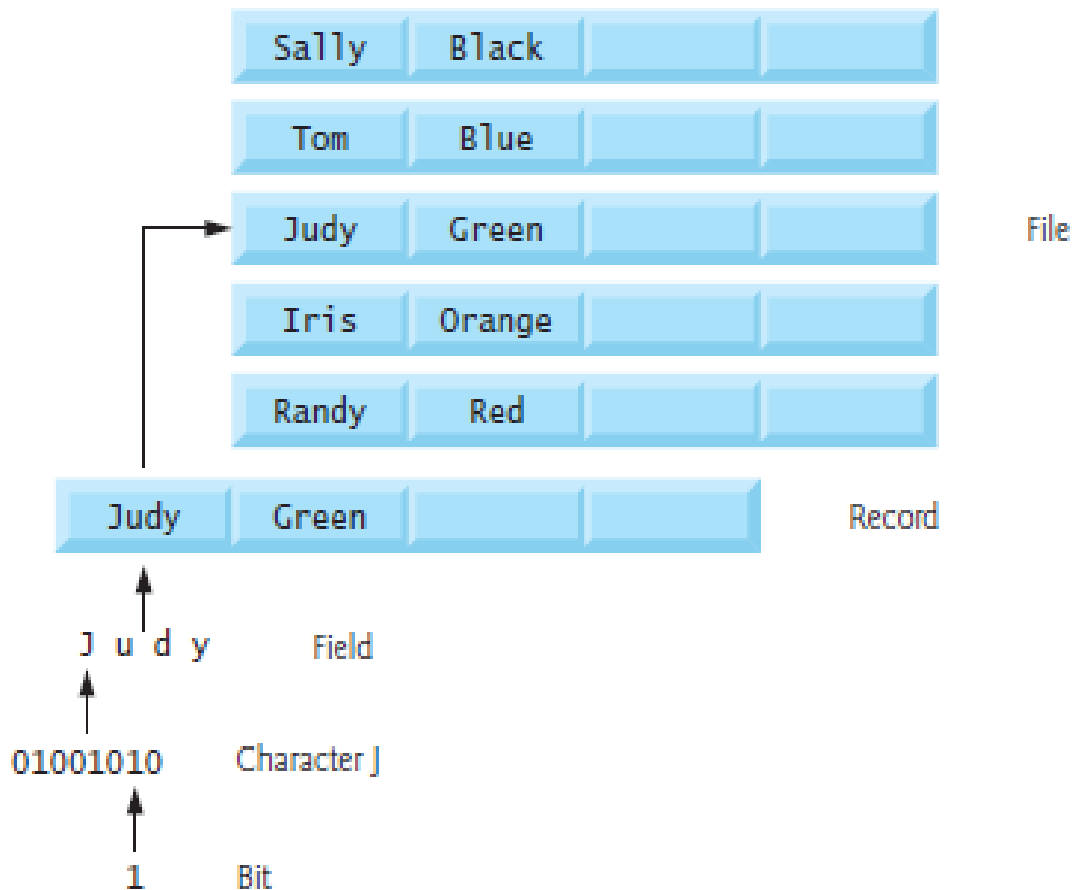
DIGITAL COMPUTERS : OUTPUT

- displayed on screens,
- printed on paper (“going green” discourages this),
- played as audio or video on smartphones, tablets, PCs and giant screens in sports stadiums,
- transmitted over the Internet, or

Information is also commonly output to secondary storage devices, such as solid-state drives (SSDs), hard drives, USB flash drives and DVD drives. Popular recent forms of output are smartphone and game-controller vibration, virtual reality devices



DIGITAL COMPUTER : DATA HIERARCHY



Bits

A bit is short for “*binary digit*”—a digit that can assume one of *two* values—and is a computer’s smallest data item. It can have the value 0 or 1.

Characters

Work with data in the low-level form of bits is tedious. Instead, people prefer to work with **decimal digits** (0–9), **letters** (A–Z and a–z) and **special symbols** such as

\$ @ % & * () - + " : ; , ? /

Digits, letters and special symbols are known as **characters**. The computer’s **character set** contains the characters used to write programs and represent data items.

MACHINE LANGUAGES: ASSEMBLY AND HIGH LEVEL LANGUAGES

- Machine languages.
- Assembly languages.
- High-level languages.

Machine Languages

- Any computer can directly understand only its own **machine language**, defined by its hardware design
- Machine languages are machine-dependent—consist of strings of numbers

+1300042774
+1400593419
+1200274027

MACHINE LANGUAGES: ASSEMBLY AND HIGH LEVEL LANGUAGES

Assembly Languages and Assemblers

- Machine languages.
 - Assembly languages.
 - High-level languages.
- English-like abbreviations to represent elementary operations.
 - Translator programs called **assemblers** were developed to convert assembly-language programs to machine language at computer speeds.

load	basepay
add	overpay
store	grosspay

MACHINE LANGUAGES: ASSEMBLY AND HIGH LEVEL LANGUAGES

High-Level Languages and Compilers

- Machine languages.
 - Assembly languages.
 - High-level languages.
- To speed the programming process, **high-level languages** were developed in which single statements could accomplish substantial tasks.
 - A typical high-level-language program contains many statements, known as the program's **source code**.
 - Translator programs called **compilers** convert high-level-language source code into machine language.

load	basepay	→	grossPay = basePay + overTimePay
add	overpay		
store	grosspay		

C is among the world's most widely used high-level programming languages.

THE C PROGRAMMING LANGUAGE

- C evolved from two earlier languages, BCPL and B
- B is used to create early versions of the UNIX operating system at Bell Laboratories in 1970
- C language was created from B by Dennis Ritchie at Bell Laboratories and was originally implemented in 1972
- Kernighan and Ritchie's book, *The C Programming Language*, was first published in 1978
- Standard C based on ANSI and ISO standards was approved in 1989
- C initially became widely known as the development language of the UNIX operating system.
- Many of today's leading operating systems are written in C and/or C++.
- C is mostly hardware-independent—with careful design, it's possible to write C programs that are **portable** to most computers.

<https://www.iso.org/standard/74528.html>

C PROGRAMMING

- **Low-Level Access:** C provides direct access to memory, pointers, and hardware registers.
- **Portability:** C code can be compiled on many architectures with minimal changes.
- **Efficiency:** C is fast and lightweight, crucial for kernel-level performance.
- **Control:** Unlike higher-level languages, C allows fine control over CPU, memory, and device management.

C PROGRAMMING

C is the primary language used to develop operating systems, due to low-level access, high performance, and portability,

OS	Language(s) Used
Linux	Mostly C, some assembly
Windows NT	Primarily C, some C++ and asm
macOS (XNU)	C, C++, Objective-C
Unix (BSDs)	Predominantly C
MINIX	C and some assembly
Embedded RTOS	Mostly C

C PROGRAMMING

- **C programming is used in web browsers**, particularly in the **core, low-level components** where performance, memory control, and system interaction are critical

Language	Role in Browser Development
C	Low-level modules, networking, protocol libraries
C++	Main language for rendering and JS engines
Rust	Used in newer components for memory safety (e.g., Firefox)
JavaScript	Runs in browser; not used to build the browser itself
Python/Go	Occasionally used in build systems or tools

C PROGRAMMING

- **C programming is used in DATABASES**, particularly in the **core, low-level components** where performance, memory control, and system interaction are critical

Database	Language Used	Notes	
SQLite	C	Entirely written in C, ultra-lightweight, widely used in mobile and embedded systems	
PostgreSQL	Mostly C	Powerful, open-source RDBMS; core engine and extensions in C	
MySQL	C and C++	Originally written in C; still uses C extensively	
Oracle DB	C	Core database engine written in C	
Berkeley DB	C	Embedded key-value store written in C	
Redis	C	In-memory key-value store for high performance	

C PROGRAMMING

- **C is extensively used in engineering**, especially in domains where **performance, low-level hardware access, and real-time processing** are crucial. It remains one of the most important languages in **embedded systems, control systems, robotics, and simulations**.
- Embedded Systems Engineering : ECUs, medical devices, industrial sensors, Arduino
- Electrical and Electronics Engineering : Device drivers, Signal Processing, FPGA processors
- Mechanical and Mechatronics Engineering : Control system implementation . Robotics, Simulations, Robot OS
- Civil and Structural Engineering : Structural Analysis Tools, simulation backends
- Physics : Simulations, Numerical solvers, CERN's ROOT framework (used in particle physics)
- Mathematics : Large scale Linear algebra problems (LAPACK, BLAS), mathematical libraries (GNU scientific library)

Arduino

C-based programming of boards

MATLAB/Simulink (C Codegen)

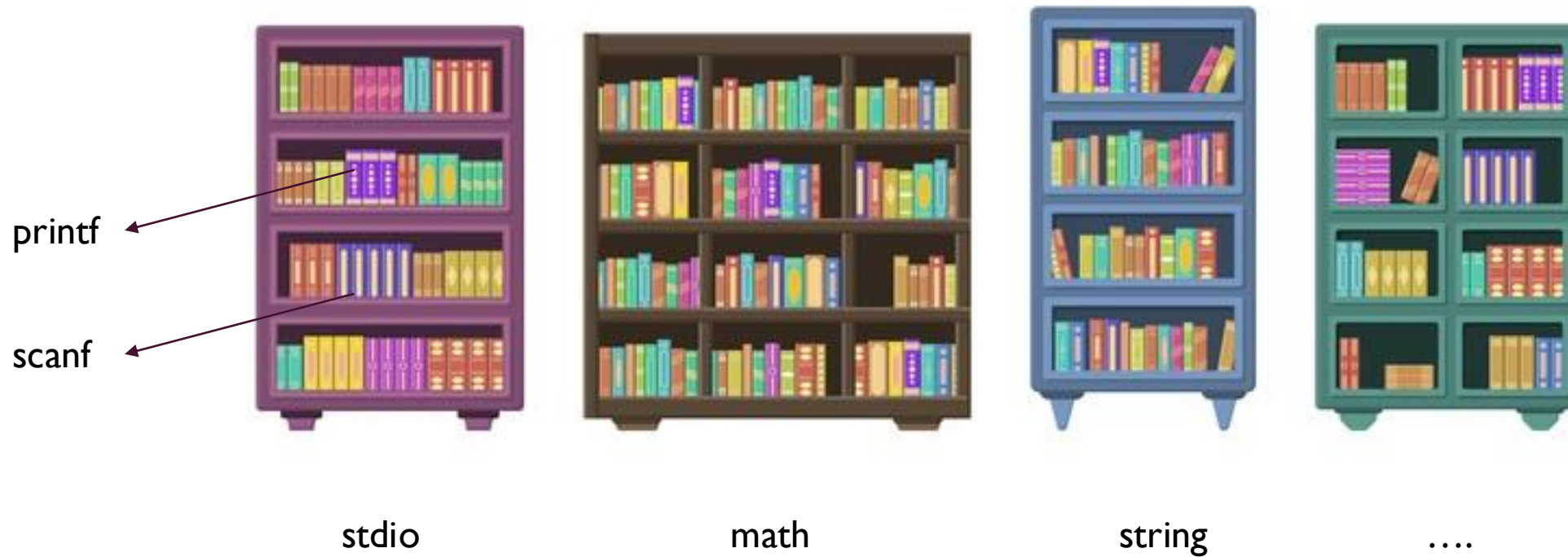
C used for auto-generated real-time code

C PROGRAM: HELLO WORLD!

```
#include <stdio.h>
```

```
int main(void) {  
    printf("Hello World!\n");  
    return 0; }
```

STANDARD LIBRARIES - C



TYPICAL C PROGRAM-DEVELOPMENT

- C programs typically go through six phases to be executed—**edit, preprocess, compile, link, load** and **execute**.
- **Phase I: Creating a Program**
- Two editors widely used on Linux systems are vi and emacs.
- C and C++ integrated development environments (IDEs) such as Microsoft Visual Studio and Apple Xcode
- have integrated editors.
- C program filenames should end with the .c extension.

TYPICAL C PROGRAM-DEVELOPMENT

- C programs typically go through six phases to be executed—**edit, preprocess, compile, link, load** and **execute**.
- **Phases 2 and 3: Preprocessing and Compiling a C Program**
- The compiler translates the C program into machine-language code
- compilation command invokes a **preprocessor** program which perform text manipulations on a program's source-code files
- A **syntax error or compile error** occurs when the compiler cannot recognize a statement in source code because it violates the C language rules.

TYPICAL C PROGRAM-DEVELOPMENT

- C programs typically go through six phases to be executed—**edit, preprocess, compile, link, load** and **execute**.
- **Phase 4: Linking**
- C programs typically use functions defined elsewhere, such as in the standard libraries, open-source libraries or private libraries of a particular project.
- The object code produced by the C compiler typically contains “holes” due to these missing parts.
- A **linker** links a program’s object code with the code for the missing functions to produce an **executable image**

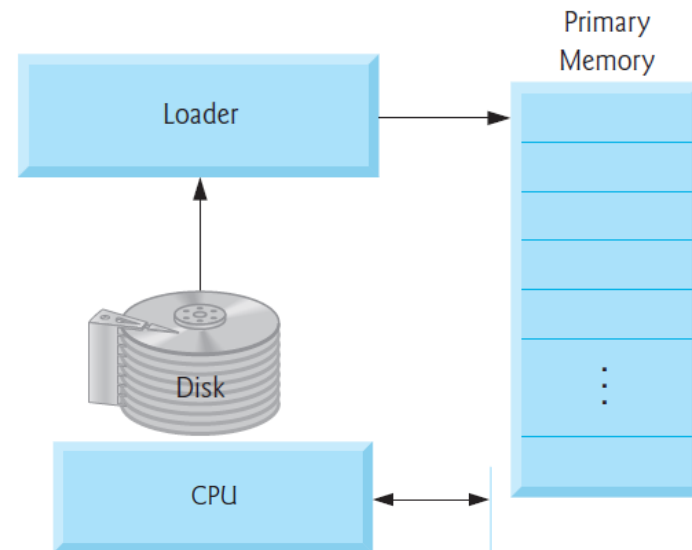
```
gcc -std=c18 welcome.c
```

If the program compiles and links correctly, the compiler produces a file named **a.out** (by default), which is **welcome.c**’s executable image

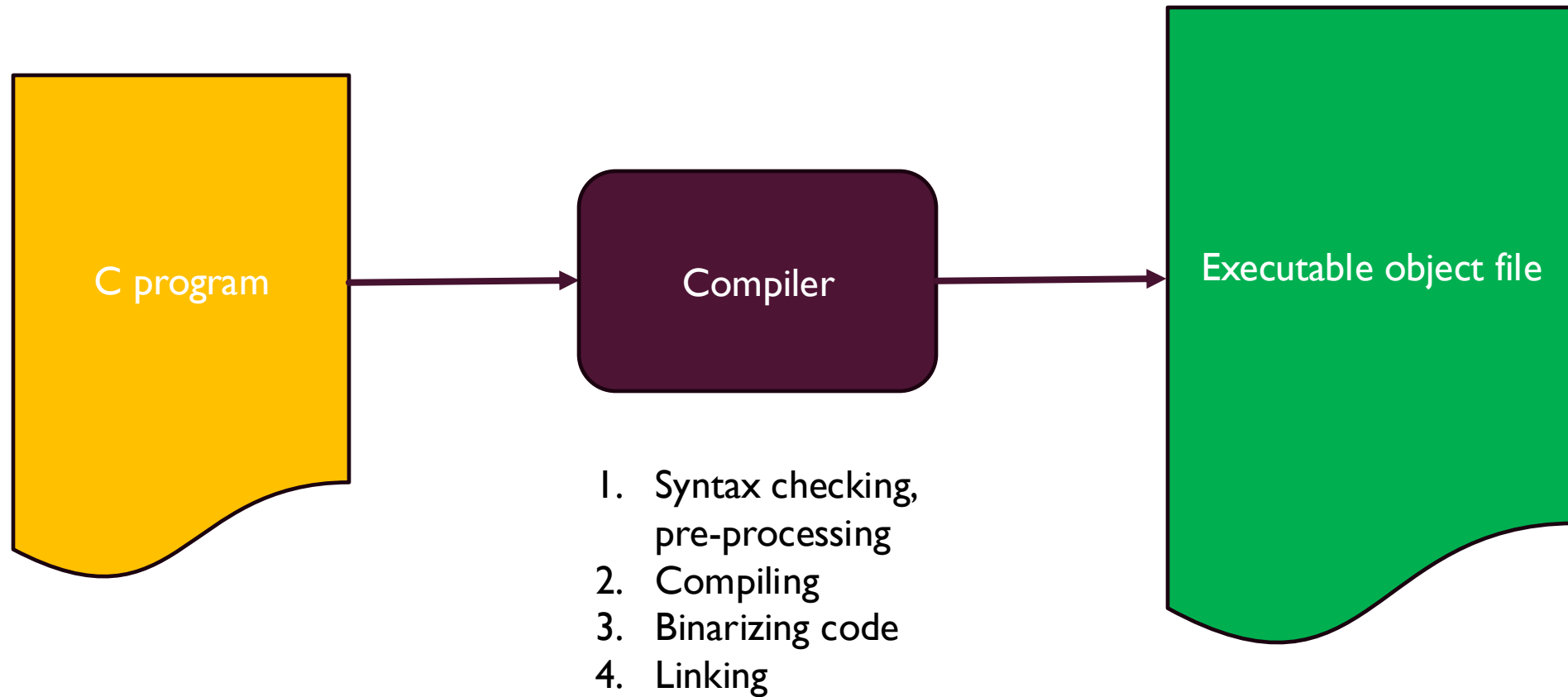
TYPICAL C PROGRAM-DEVELOPMENT

- C programs typically go through six phases to be executed—**edit**, **preprocess**, **compile**, **link**, **load** and **execute**.
- **Phases 5: Loading and Execution**
- The **loader** takes the executable image from disk and transfers it to memory, and CPU **executes** the program one instruction at a time

./a.out



PIPELINE



FURTHER READING...

- Sections 1.1, 1.2, 1.5-1.9 in Deitel & Deitel book